

bok25

基
礎
班

、考研数据結構



数据结构万用法则

了解逻辑结构→分析其存储结构→学习其基本操作



了解逻辑结构：线性表是什么？

1. 线性表是一种逻辑结构

2. 表中元素的个数有限

3. 表中数据元素类型相同

4. 除第一个元素外，每个数据元素均只有一个前驱

5. 除最后一个元素外，每个数据元素均只有一个后继



(连线奥迪罢了)



线性表的类型定义

ADT List{

数据对象 : $D = \{a_i | a_i \in \text{ElemSet}, i = 1, 2, \dots, n, n \geq 0\}$

数据关系 : $R = \{ \langle a_{i-1}, a_i \rangle | a_{i-1}, a_i \in D, i = 2, \dots, n \}$

基本操作 :

InitList(&L)

操作结果 : 构造一个空的线性表L。

DestroyList(&L)

初始条件 : 线性表L已存在。

操作结果 : 销毁线性表L。

ClearList(&L)

初始条件 : 线性表L已存在。

操作结果 : 将L重置为空表。

ListEmpty(L)

初始条件 : 线性表L已存在。

操作结果 : 若L为空表, 则返回true, 否则返回false。

ListLength(L)

初始条件 : 线性表L已存在。

操作结果 : 返回L中数据元素的个数。

GetElem(L, i, &e)

初始条件 : 线性表L已存在, 且 $1 \leq i \leq \text{ListLength}(L)$ 。

操作结果 : 用e返回L中第i个数据元素的值。

LocateElem(L, e)

初始条件 : 线性表L已存在。

操作结果 : 返回 L 中第 1 个值与 e 相同的元素在 L 中的位置。若这样的数据元素不存在, 则返回值为 0。

PriorElem(L, cur_e, &pre_e)

初始条件 : 线性表L已存在。

操作结果 : 若cur_e是L的数据元素, 且不是第一个, 则用pre_e返回其前驱, 否则操作失败, pre_e 无定义。

NextElem(L, cur_e, &next_e)

初始条件 : 线性表L已存在。

操作结果 : 若cur_e是L的数据元素, 且不是最后一个, 则用next_e返回其后继, 否则操作失败, next_e 无定义。

ListInsert(&L, i, e)

初始条件 : 线性表L已存在, 且 $1 \leq i \leq \text{ListLength}(L) + 1$ 。

操作结果 : 在L中第i个位置之前插入新的数据元素e, L的长度加1。

ListDelete(&L, i)

初始条件 : 线性表L已存在且非空, 且 $1 \leq i \leq \text{ListLength}(L)$ 。

操作结果 : 删除L的第i个数据元素, L的长度减1。

TraverseList(L)

初始条件 : 线性表L已存在。

操作结果 : 对线性表L进行遍历, 在遍历过程中对L的每个节点访问一次。

}ADT List



线性表的类型定义（考试版）

不这么写！

```
Status InitList(SqList &L)
{
    // 构造一个空的顺序表 L
    L.elem=new ElemType[MAXSIZE]; //为顺序表分配一个大小为MAXSIZE的数组空间
    if(!L.elem) exit(OVERFLOW); //存储分配失败退出
    L.length=0; //空表长度为0
    return OK;
}
```

该这么写！

```
InitList(&L)
```



正式开始之前，请不要混淆

线性表 $\longrightarrow$ 逻辑结构

顺序表 $\longrightarrow$ 线性表的顺序存储

顺序存储下的线性表

存储地址	内存状态	位序
b	A1	1
$b+s$	A2	2
...	...	...
$b+(i-1)s$	Ai	i
...	...	...
$b+(n-1)s$	An	n
$b+ns$	空	空
...	...	
$b+(\text{maxlen}-1)s$	空	

```
//- - - - - 顺序表的存储结构- - - - -
typedef struct{
    ElemType *elem;    //存储空间的基地址
    int length;        //当前长度
}SqList;              // 顺序表的结构类型为 SqList
```

随机存取

存储地址	内存状态	位序
b	A1	1
b+s	A2	2
...	...	...
b+(i-1)s	Ai	i
...	...	...
b+(n-1)s	An	n
b+ns	空	} 空
...	...	
b+(maxlen-1)s	空	

数组

A[0]	A[1]	A[2]	A[3]	A[4]
------	------	------	------	------

位序

1

2

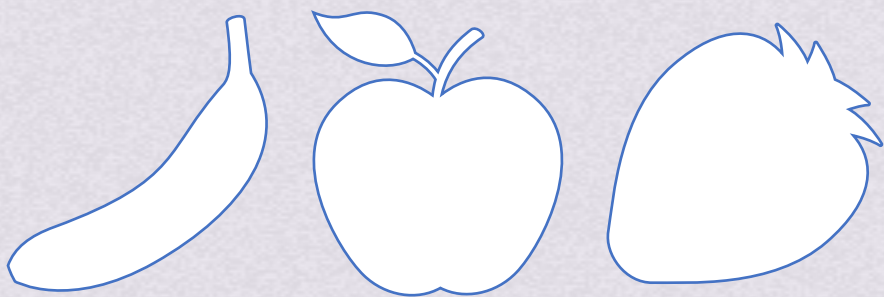
3

4

5



结构体



```
struct Fruit{  
    ElemType Apple;  
    ElemType Banana;  
    ElemType Strawberry;  
}
```

```
typedef struct Fruit{  
    ElemType Apple;  
    ElemType Banana;  
    ElemType Orange;  
}Fru;
```

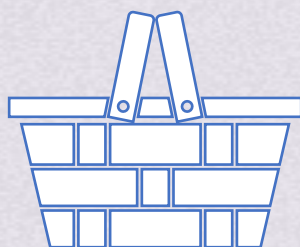
申请果篮a和b

Fruit a;
Fruit b;

Fru a;
Fru b;



结构体



```
struct Fruit{  
    ElemType Apple;  
    ElemType Banana;  
    ElemType Strawberry;  
}
```

```
typedef struct Fruit{  
    ElemType Apple;  
    ElemType Banana;  
    ElemType Orange;  
}Fru;
```

申请果篮a和b

```
Fruit a;  
Fruit b;
```

```
Fru a;  
Fru b;
```

```
//- - - - - 顺序表的存储结构- - - - -
typedef struct{
    ElemType *elem;    //存储空间的基地址
    int length;    //当前长度
}SqList;    // 顺序表的结构类型为 SqList
```

存储地址	内存状态	位序
b	A1	1
$b+s$	A2	2
...	...	...
$b+(i-1)s$	Ai	i
...	...	...
$b+(n-1)s$	An	n
$b+ns$	空	空
...	...	
$b+(maxlen-1)s$	空	



三句话，让顺序表主动告诉我其长度

```
InitList(&L);  
int a=L.length;  
return a;
```



基本操作合集（创建）

```
Status InitList(SqList &L)
{
    //构造一个空的顺序表 L
    L.elem=(Elemtype*)malloc(sizeof(Elemtype)*InitSize); //为顺序表分配一个大小为Initsize的数组空间
    if(!L.elem) exit(OVERFLOW);                          //存储分配失败退出
    L.length=0;                                           //空表长度为0
    return OK;
}
```

新创建节点L.elem



基本操作具体实现（取值）

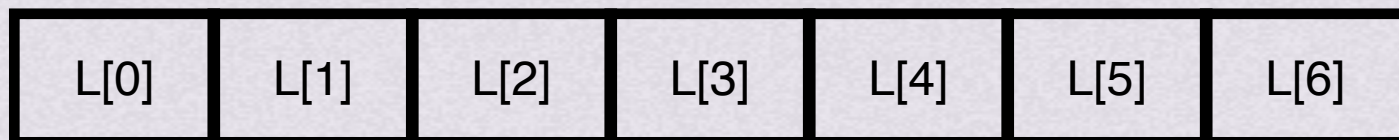
```
Status GetElem(SqList L, int i, ElemType &e){  
    if(i < 1 || i > L.length) return error; //判断i的值是否合理, 不合理error  
    e = L.elem[i - 1] //elem[i-1]单元存储第i个数据元素  
    return ok;  
}
```





基本操作具体实现（按值查找）

```
int LocateElem(SqList L, ElemType e)
{ // 在顺序表 L 中查找值为 e 的数据元素，返回其序号
  for (i = 0; i < L.length; i++)
  {
    if (L.elem[i] == e) {
      return i + 1; // 查找成功，返回序号 i+1
    }
  }
  return 0; // 查找失败，返回 0
}
```





基本操作具体实现（插入）

```
Status ListInsert(SqList &L, int i, ElemType e){  
    if((i<1) || (i>L.length+1)){//如果i的值不合法, 返回error  
        return error;  
    }  
    if(L.length==MAXSIZE) return error;//如果存储空间满, 返回error  
    for(j=L.length-1; j>=i-1; j--){  
        L.elem[j+1]=L.elem[j];}           //插入位置之后的元素后移  
    L.elem[i-1]=e;                        //新元素e放入第i个位置  
    ++L.length;  
    return ok;  
}
```



基本操作具体实现（删除）

```
Status ListDelete(SqList &L, int i)
{
    // 在顺序表L中删除第i个元素，i值的合法范围是  $1 \leq i \leq L.length$ 
    if((i < 1) || (i > L.length)) return ERROR; // i值不合法
    for(j = i; j <= L.length - 1; j++) {
        L.elem[j - 1] = L.elem[j]; // 被删除元素之后的元素前移
    }
    --L.length; // 表长减1
    return OK;
}
```